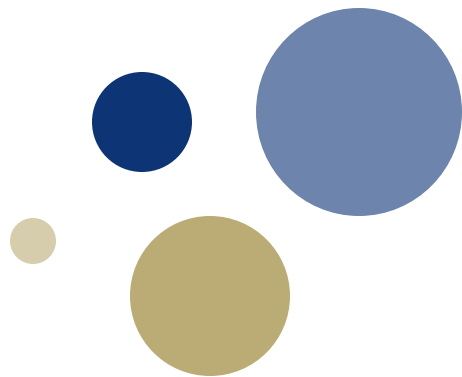




NTNU

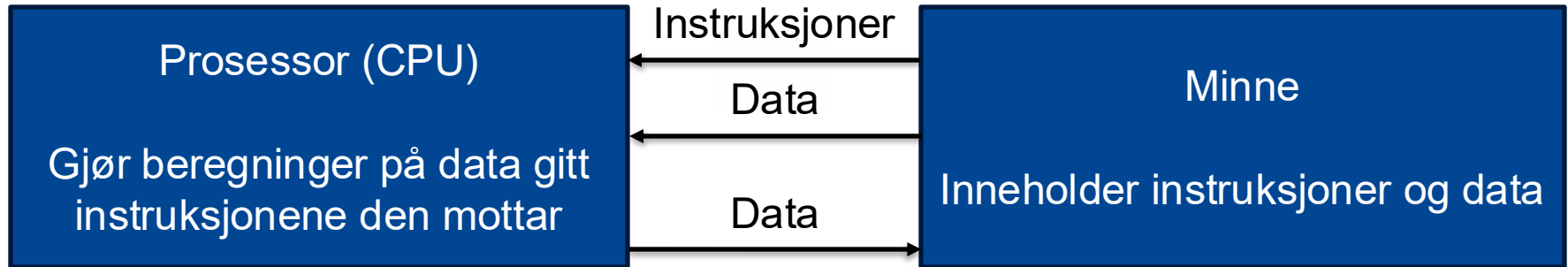
Kunnskap for en bedre verden

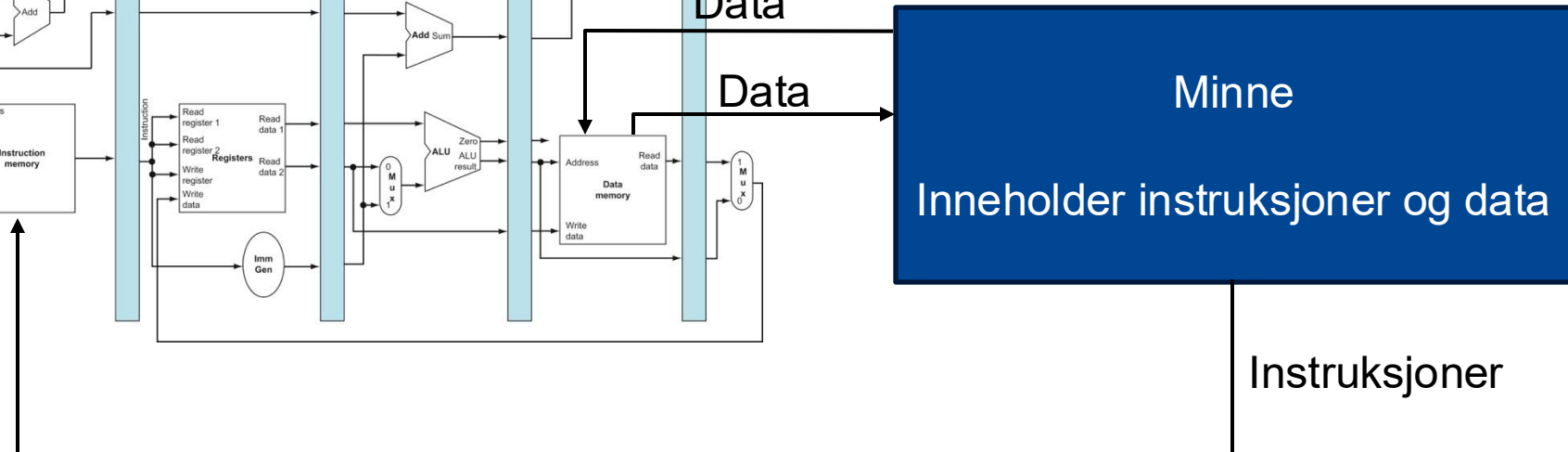


Forelesning 10: Minneteknologi, bedre hurtigbuffer, og virtuelt minne

TDT4160 Datamaskiner

Prinsippet om lagrede program

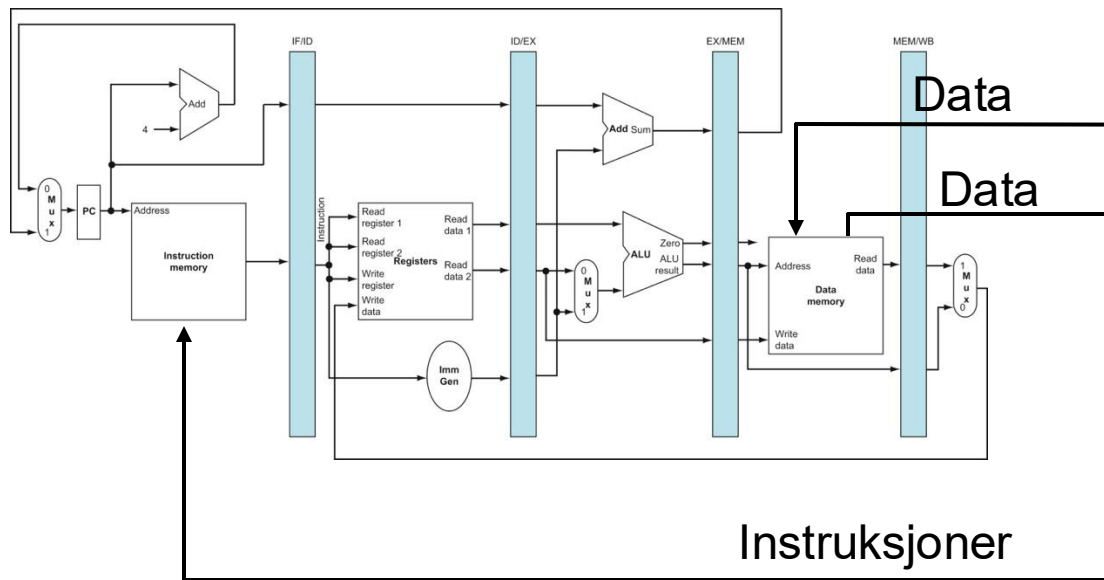




Prinsippet om lagrede program

Prosessorbrikken

Prossessor



I dag: Minneteknologier, raskere hurtigbuffer, og virtuelt minne

Datahurtigbuffer

Minne

Instruksjonshurtigbuffer



Tema 6.2: Minneteknologier (Kapittel 5.2 og A.9)

MINNETEKNOLOGIER

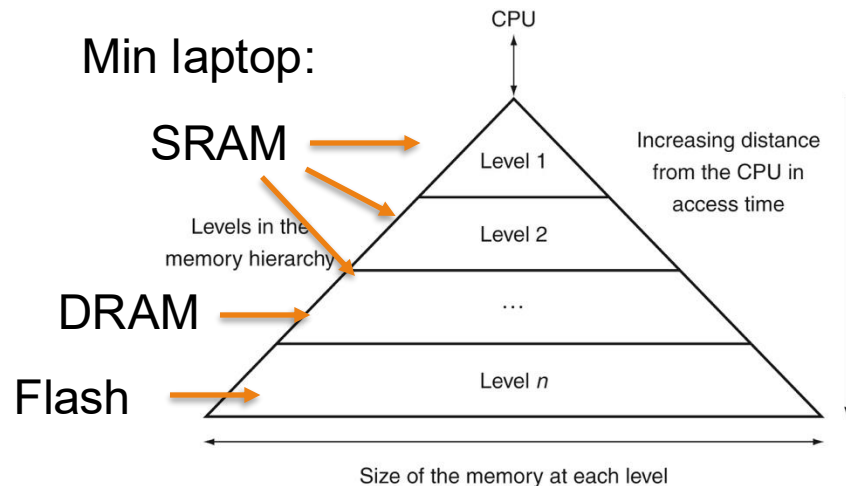
Læringsutbytte T6.2

- Studenten skal kunne forklare begrepene **volatilt og ikke-volatilt minne** samt **statisk og dynamisk minne**.
- Studenten skal kjenne til omtrentlig **aksesstid** og **kostnad per bit** for **SRAM**, **DRAM**, **Flash**, og **magnetisk disk** og kunne forklare hvordan dette påvirker hvordan teknologiene brukes i minnehierarkiet.
- Studenten skal kunne **konstruere** et **SRAM minne** fra grunnleggende kombinatoriske og sekvensielle komponenter (kunne gjengi og forklare figur A.9.3).
- Studenten skal kunne beskrive **prinsippene** for hvordan vi konstruerer **DRAM minner** (kunne gjengi og forklare figur A.9.5 og A.9.6).

Volatile og ikke-volatile minner

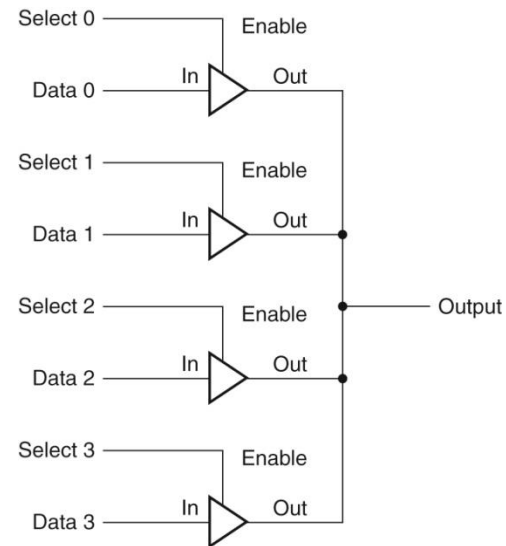
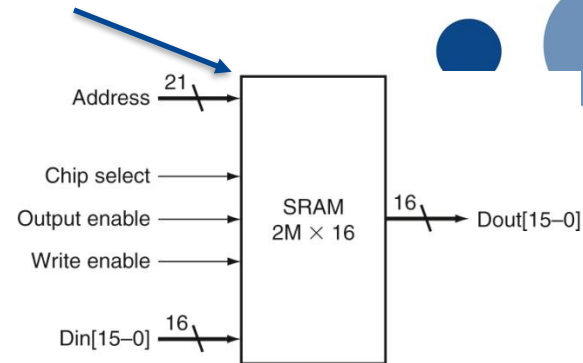
- Volatile minnetyper mister verdiene når vi skruer av strømmen
 - SRAM: «Static Random Access Memory»
 - DRAM: «Dynamic Random Access Memory»
- Ikke-volatile minnetyper beholder verdiene når vi skruer av strømmen
 - Flash
 - Magnetiske harddisker

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12
Magnetic disk	5,000,000–20,000,000 ns	\$0.01–\$0.02



SRAM (1)

- «Random Access Memory»
 - ... betyr at det tar like lang tid å aksessere alle adresser
- «Static RAM»
 - Lagrer verdiene i låser
 - Verdiene beholdes så lenge strømmen er på (statisk)
- Det var da veldig likt en registerfil?
 - ... men registerfila skalerer ikke fordi den bruker en multiplekser per utgang
 - En 32-til-1 multiplekser er OK, men en 2097152-til-1 multiplekser er ikke OK (som vi trenger til minnet til høyre)
- Løsning: «Tristate buffer»
 - Slipper enten inngangsverdien igjennom til utgangen eller kobler seg fra utgangen («high impedance state»)
 - Gjør det mulig å koble flere minneelementer til samme utgang!



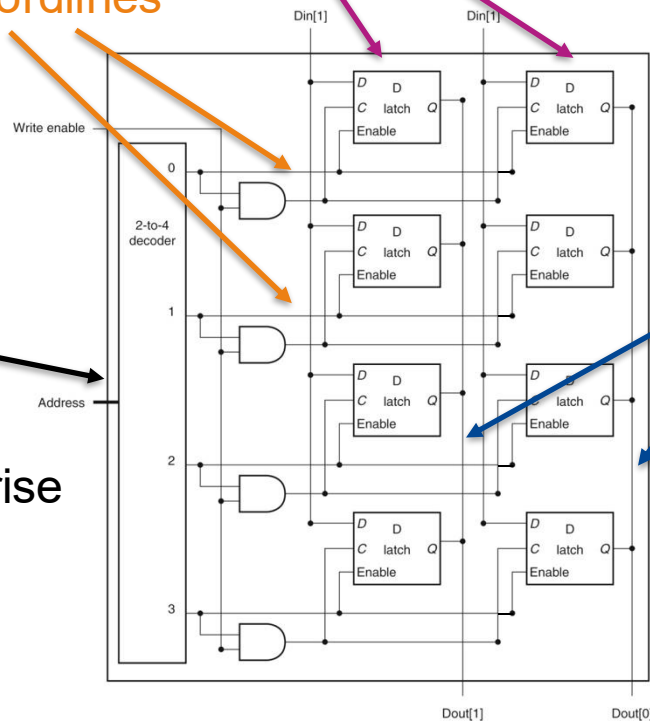
SRAM (2)

Alle låser har tristate buffer på Q-utgangen

... men blir ikke dekoderen også stor?

Løsning: Organiser minnet som en matrise

Wordlines

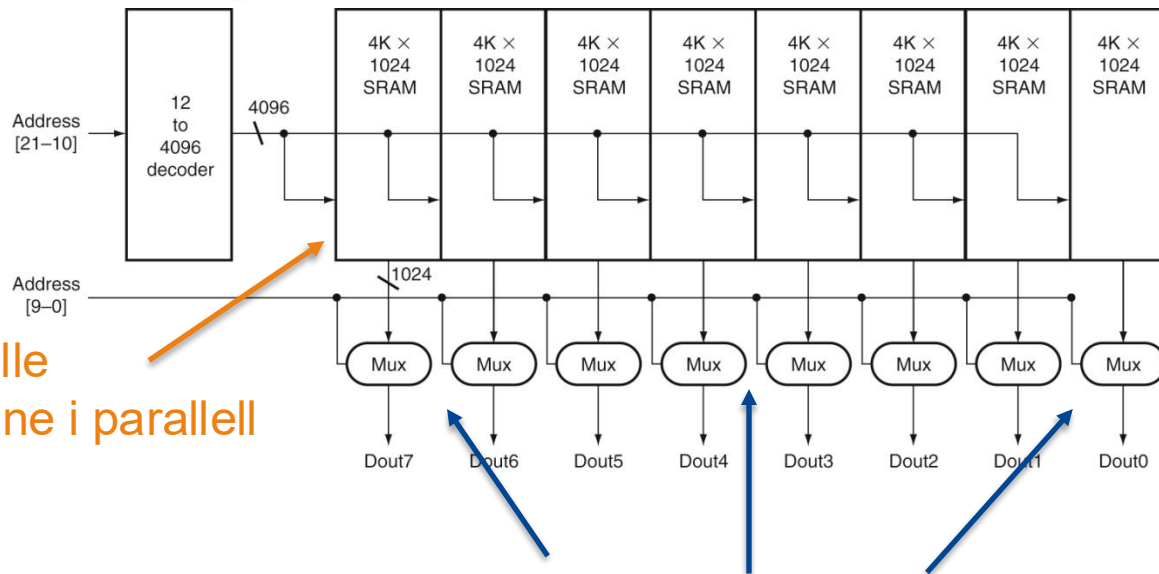


Bitlines

Eksempel: En 4x2 SRAM (Figur A.9.3)

SRAM (3)

Dekoderen tar seg av noen av
adressebitene



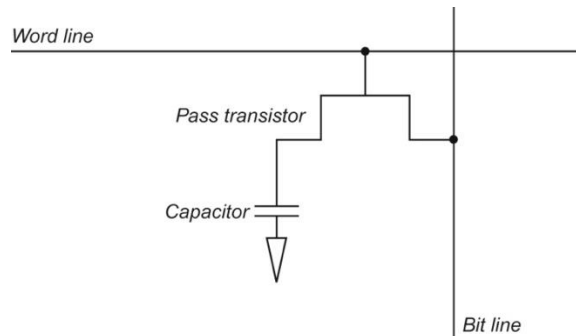
Vi leser fra alle
minneblokkene i parallell

Eksempel:
En 4Mi x 8 SRAM

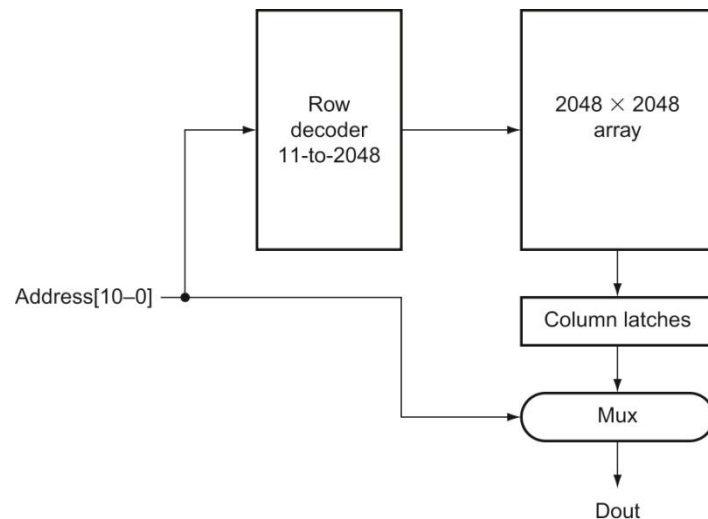
Hver minneblokk leverer ett bit som velges ut av
multipleksere basert på resten av adressen

DRAM

- DRAM lagrer verdier i kondensatorer
 - Høyere tetthet enn SRAM og dermed billigere per bit
 - ... men mister ladningen over tid (dynamisk)
 - Løsning: Lese verdiene ut og skrive de tilbake før de forsvinner («refresh»)
- Kondensatoren lades ut når vi leser
 - ... og derfor må vi ha låser på utgangen og skrive raden tilbake igjen etterpå
- Adressen sendes i to skritt:
 - Adressen til raden («Row Access Strobe (RAS)»)
 - Adressen til kolonnen («Column Access Strobe (CAS)»)
 - Vi kan aksessere neste verdi raskt ved å bare sende en ny kolonneadresse!



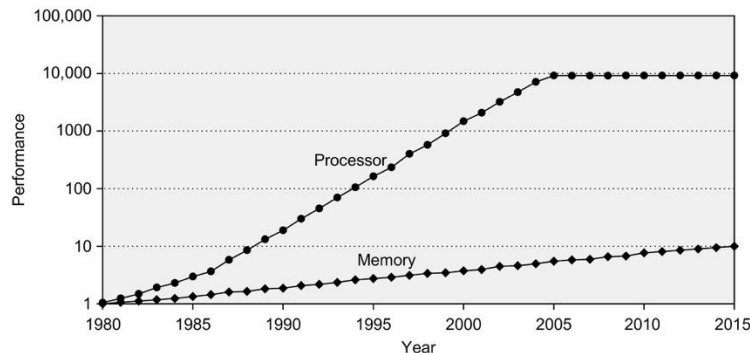
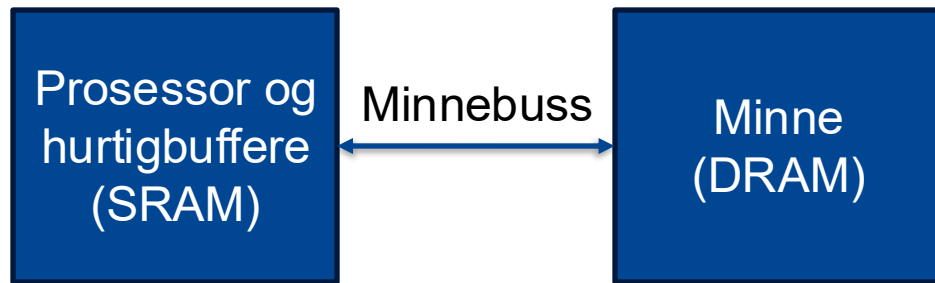
Figur A.9.5



Figur A.9.6

Minnebuss

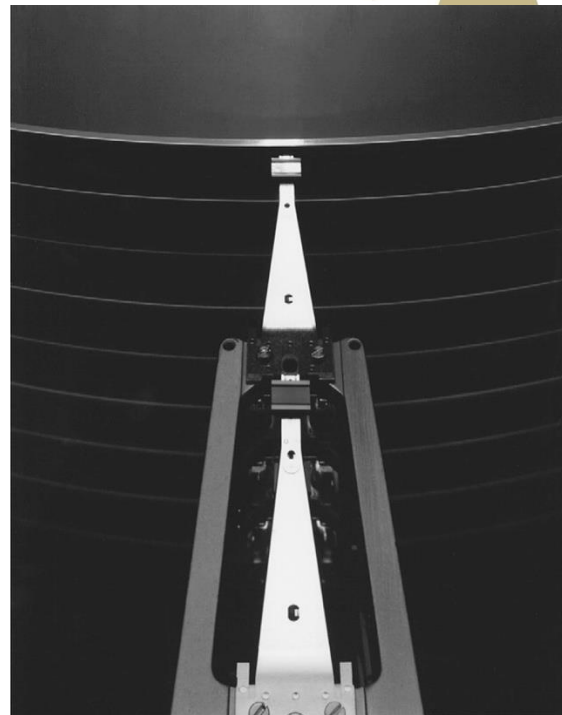
- SRAM/logikk bruker ulike produksjonsprosesser
 - SRAM/logikk-prosessen er optimalisert for raske transistorer
 - DRAM-prosessen er optimalisert for høy tetthet for kondensatorene
- Prosessor og hurtigbuffer er dermed typisk på en brikke mens DRAM er på andre brikker
 - Minnebussen kobler disse sammen og er designet for høy båndbredde
 - Eksempler: DDR5, GDDR5, HBM2, etc.
 - DDR = «Double Data Rate»: Sender data på både stigende og synkende klokkeflanke



«The Memory Wall»

Flash og magnetisk disk

- Flash
 - En EEPROM («Electrically Erasable Programmable Read-Only Memory»)
 - Hvert bit kan bare skrives et gitt antall ganger før det blir utslitt
 - Løsning: Fordel skriveingen utover hele minnet («wear leveling»)
- Magnetisk disk
 - Hvert bit er lagret i et magnetisk materiale på en snurrende plate
 - Aksess krever at vi:
 - Flytter lesehodet til riktig spor
 - Venter til vi har rotert til riktig sektor
 - Gjennomføre lese eller skriveoperasjonen
 - Ikke «random access»: Det tar lengre tid å lese noen verdier enn andre
 - Det lønner seg å lese mye data når du først har fått lesehodet til riktig sted





Tema 6.1: Minnehierarki og hurtigbuffer (Kapittel 5.3 og 5.4)

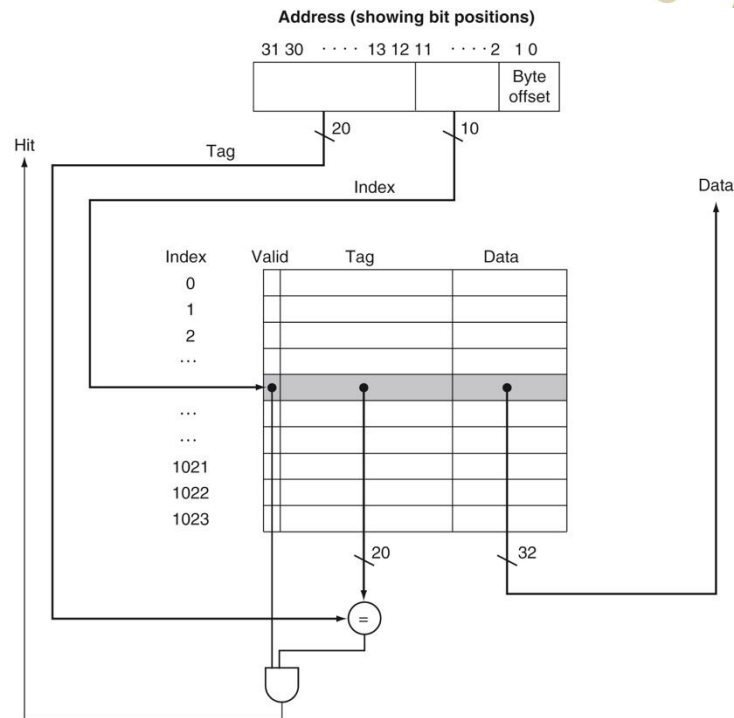
HURTIGBUFFER: BLOKKSTØRRELSE

Læringsutbytte T6.1

- Studenten skal kunne forklare begrepene lokalitet i tid og lokalitet i rom og vite hvorfor lokalitet oppstår.
- Studenten skal forstå hvordan og hvorfor et hierarki av minner kan gi illusjonen av ett stort og raskt minne.
- Studenten skal forstå hvordan vi **konstruerer et direkteilordnet hurtigbuffer** (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.
- Studenten kunne forklare hvordan hurtigbufferne kan integreres i en samlebåndsarkitektur.
- Studenten skal kjenne til prinsippene for hvordan hurtigbufferne håndterer skriveoperasjoner.
- Studenten skal kunne beregne hvordan minneaksesstid påvirker datamaskinens ytelse.
- Studenten skal kunne beregne hvordan treffraten i flernivå hurtigbuffer påvirker den gjennomsnittlige aksessiden.
- Studenten skal forstå hvorfor settassosiative og fullassosiative hurtigbufferne kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).
- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke treffraten i hurtigbufferne.

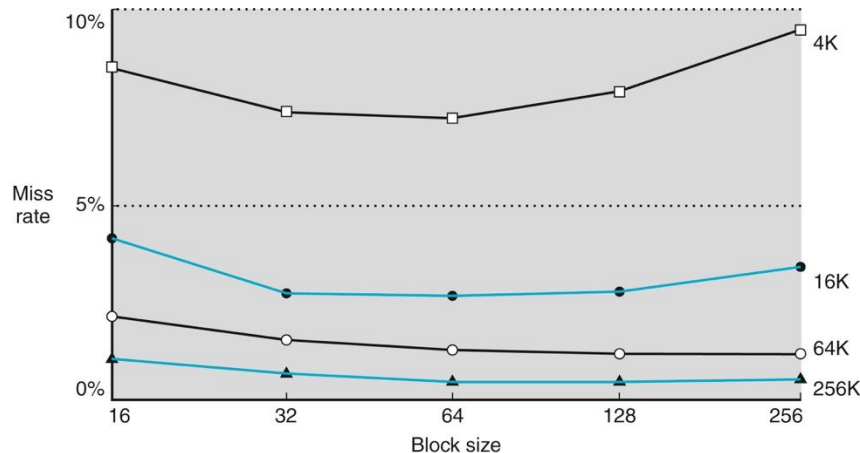
Et direktetilordnet hurtigbuffer

- Vi bruker bit 11-2 til indeks fordi sekvensiell aksess er vanlig
- Hver blokk må ha et «valid» bit
 - Valid = 1: Verdien er gyldig
 - Vi trenger disse ved oppstart og hvis vi må tømme hurtigbufferet



Hvor stor bør en blokk være?

- En hurtigbufferblokk kan være større enn ordbredden til prosessoren
- Større blokker gir:
 - Fordel: Bedre utnyttelse av lokalitet i rom
 - Ulempe: Plass til færre blokker i hurtigbufferet
 - Ulempe: Kostnaden ved å hente nye data øker

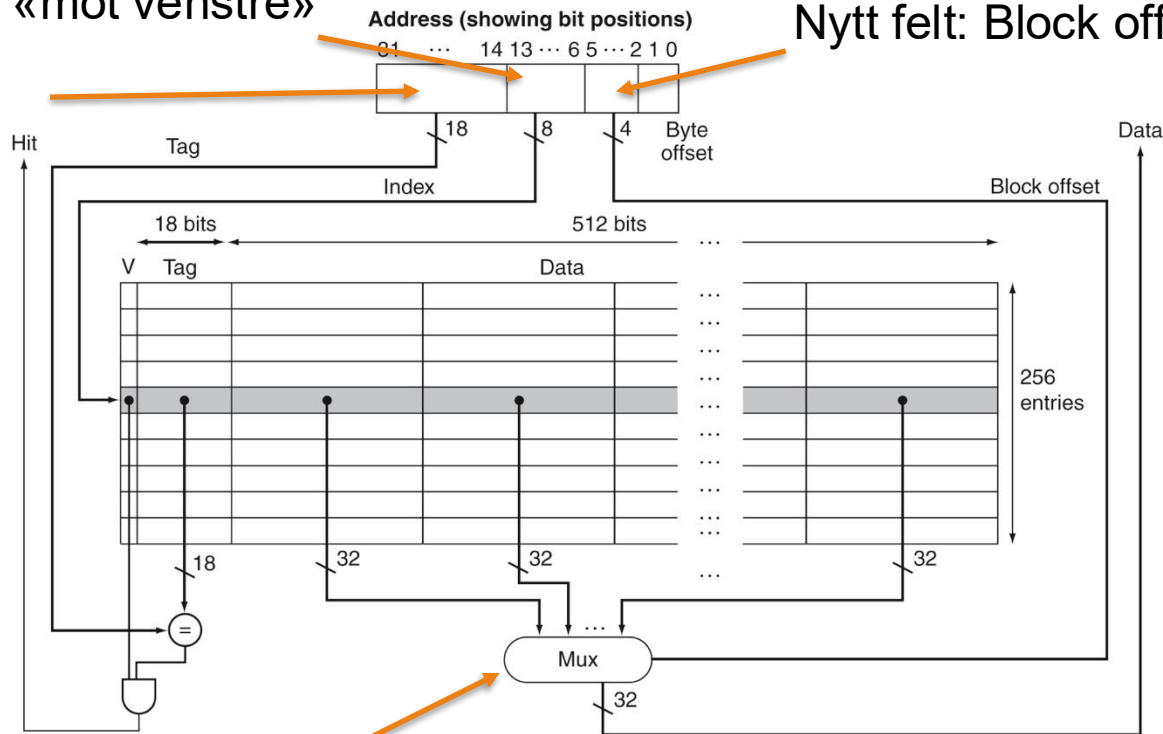


Flere ord i en blokk

Index er flyttet «mot venstre»

Færre bit i tag

Nytt felt: Block offset



Vi trenger en multiplekser for å hente rett ord

Eksempel: 8 byte hurtigbufferblokker

Repetisjon: 4 byte blokker

Adresseformat

Tag				Indeks		Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (16 byte)

Data		Tag	
Indeks	Verdi	Indeks	Verdi
3 (11)		3 (11)	
2 (10)		2 (10)	
1 (01)		1 (01)	
0 (00)		0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (binær)	Verdi (32 bit)
1111 11 00	-
...	...
0001 11 00	376
0001 10 00	375
0001 01 00	-
0001 00 00	-
0000 11 00	258
0000 10 00	42
0000 01 00	-
0000 00 00	-

Eksempel: 8 byte hurtigbufferblokker

Nå med 8 byte blokker

Adresseformat

Tag			Indeks		B	Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (32 byte)

Nytt format
Større
hurtigbuffer
(men det må det
ikke være)

Data

Indeks	Verdi 1	Verdi 0
3 (11)		
2 (10)		
1 (01)		
0 (00)		

Tag

Indeks	Verdi
3 (11)	
2 (10)	
1 (01)	
0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (binær)	Verdi (32 bit)
1111 11 00	-
...	...
0001 11 00	376
0001 10 00	375
0001 01 00	-
0001 00 00	-
0000 11 00	258
0000 10 00	42
0000 01 00	-
0000 00 00	-

Eksempel: 8 byte hurtigbufferblokker

Les adresse 24 (0001 1000)

Adresseformat

Tag			Indeks		B	Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (32 byte)

Data			Tag	
Indeks	Verdi 1	Verdi 0	Indeks	Verdi
3 (11)			3 (11)	
2 (10)			2 (10)	
1 (01)			1 (01)	
0 (00)			0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (binær)	Verdi (32 bit)
1111 11 00	-
...	...
0001 11 00	376
0001 10 00	375
0001 01 00	-
0001 00 00	-
0000 11 00	258
0000 10 00	42
0000 01 00	-
0000 00 00	-

Eksempel: 8 byte hurtigbufferblokker

Les adresse 12 (0000 1100)

Adresseformat

Tag			Indeks		B	Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (32 byte)

Data			Tag	
Indeks	Verdi 1	Verdi 0	Indeks	Verdi
3 (11)	376	375	3 (11)	000
2 (10)			2 (10)	
1 (01)			1 (01)	
0 (00)			0 (00)	

Minne ($2^8 = 256$ byte)

Adresse (binær)	Verdi (32 bit)
1111 11 00	-
...	...
0001 11 00	376
0001 10 00	375
0001 01 00	-
0001 00 00	-
0000 11 00	258
0000 10 00	42
0000 01 00	-
0000 00 00	-

Eksempel: 8 byte hurtigbufferblokker

Les adresse 8 (0000 1000)

Adresseformat

Tag			Indeks		B	Offset	
7	6	5	4	3	2	1	0

Hurtigbuffer (32 byte)

Data		
Indeks	Verdi 1	Verdi 0
3 (11)	376	375
2 (10)		
1 (01)	258	42
0 (00)		

Tag	
Indeks	Verdi
3 (11)	000
2 (10)	
1 (01)	000
0 (00)	

Vi treffer i
hurtigbufferet selv
om vi ikke har
aksessert adresse
8 før!

Minne ($2^8 = 256$ byte)

Adresse (binær)	Verdi (32 bit)
1111 11 00	-
...	...
0001 11 00	376
0001 10 00	375
0001 01 00	-
0001 00 00	-
0000 11 00	258
0000 10 00	42
0000 01 00	-
0000 00 00	-



Tema 6.1: Minnehierarki og hurtigbuffer (Kapittel 5.3 og 5.4)

HURTIGBUFFER: HØYERE YTELSE

Læringsutbytte T6.1

- Studenten skal kunne forklare begrepene lokalitet i tid og lokalitet i rom og vite hvorfor lokalitet oppstår.
- Studenten skal forstå hvordan og hvorfor et hierarki av minner kan gi illusjonen av ett stort og raskt minne.
- Studenten skal forstå hvordan vi konstruerer et direktetilordnet hurtigbuffer (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.
- Studenten kunne forklare hvordan hurtigbufferne kan integreres i en samlebåndsarkitektur.
- Studenten skal kjenne til prinsippene for hvordan hurtigbufferne håndterer skriveoperasjoner.
- Studenten skal kunne **beregne** hvordan **minneaksesstid** påvirker datamaskinens **ytelse**.
- Studenten skal kunne **beregne** hvordan **treffraten** i flernivå hurtigbuffer påvirker den gjennomsnittlige aksessiden.
- Studenten skal forstå hvorfor **settassosiative** og **fullassosiative hurtigbufferne** kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).
- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke **treffraten** i **hurtigbufferne**.

Ytelse og minnesystemet

- Hvis vi skal ha en datamaskin med høy ytelse, må vi ha et minnesystem med høy ytelse
 - Amdahl's lov
- Hvordan påvirker minnesystemet ytelsen til et program?
 - For å finne ut av dette kan vi se på CPI-komponenten som minnesystemet er ansvarlig for

CPI = Cycles Per Instruction

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

$$CPI = \frac{C_{Beregning} + C_{Minne}}{I_{Beregning} + I_{Minne}}$$

$$= \frac{C_{Beregning}}{I_{Beregning} + I_{Minne}} + \frac{C_{Minne}}{I_{Beregning} + I_{Minne}}$$

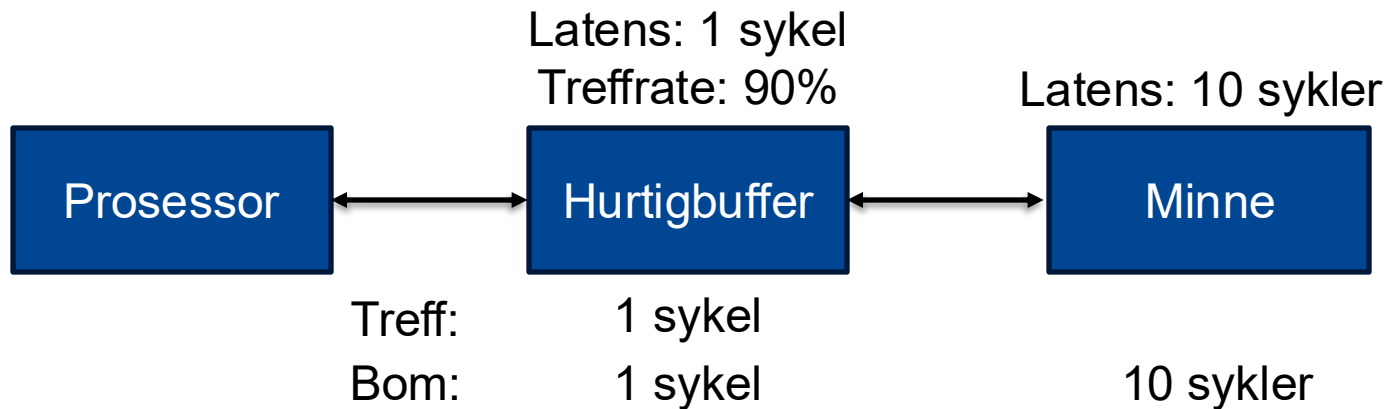
$$= \frac{C_{Beregning}}{I_{Total}} + \frac{C_{Minne}}{I_{Total}}$$

Vi må gjøre denne så liten som mulig!

Minnelatens

$$CPI = \frac{C_{Beregning}}{I_{Total}} + \frac{C_{Minne}}{I_{Total}}$$

- Minnelatens = Antallet klokkesykler det tar å hente en verdi fra en minneenhet
- Vi kan dermed se på C_{Minne}/I_{Total} som den gjennomsnittlige minnelatensen over alle instruksjoner
- Vi kjenner antallet instruksjoner, så det er C_{Minne} vi må måle/estimere
 - Hvis vi deler C_{Minne} på antall minneforespørsler får vi «Average Memory Access Latency (AMAT)»



$$AMAT = 0.9 \cdot 1 + 0.1 \cdot (1 + 10) = 2$$

Beregnings- og minneintensitet

- Beregningsintensiv
 - Programmet bruker en relativt **stor** andel av kjøretiden sin på beregning
 - $I_{Beregning}$ er (mye) **større** enn I_{Minne}
 - ... og/eller C_{Minne} er **lav**
- Minneintensiv
 - Programmet bruker en relativt **liten** andel av kjøretiden sin på beregning
 - $I_{Beregning}$ er (mye) **mindre** enn I_{Minne}
 - ... og/eller C_{Minne} er **høy**
- Jo mer minneintensiv applikasjonen er, jo mer påvirker minnelatensen ytelse

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

CPI = Cycles Per Instruction

$$CPI = \frac{C_{Beregning} + C_{Minne}}{I_{Beregning} + I_{Minne}}$$

Hvordan kan vi øke ytelsen til minnesystemet?

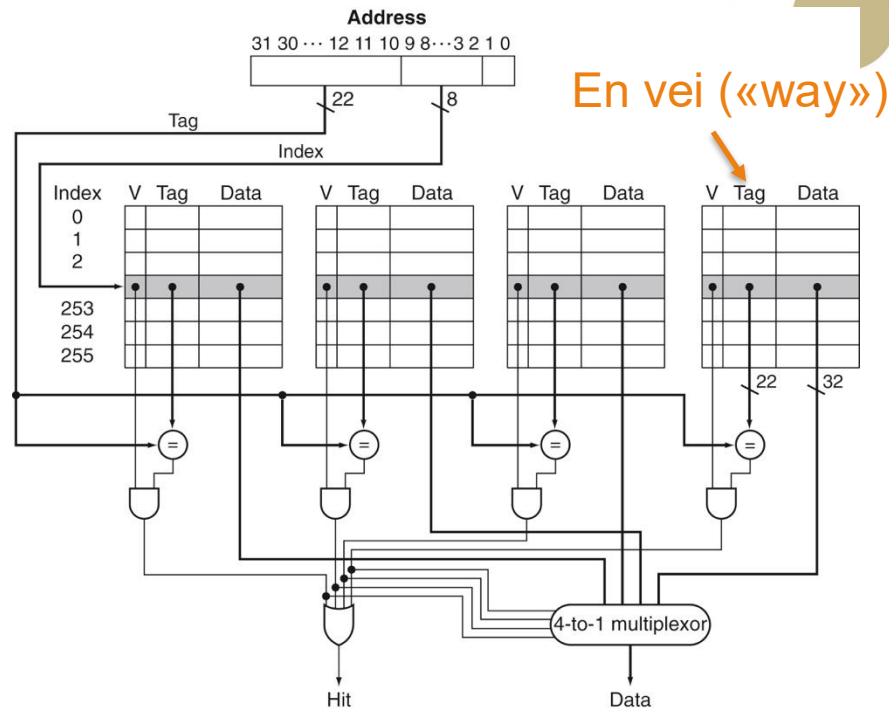


- To strategier:
 - (1) Øke treffraten i hurtigbufferet
 - (2) Redusere tiden det tar å hente data
- Vi skal se på:
 - Sett- og full-assosiative hurtigbuffer (Strategi 1 i maskinvare)
 - Flernivå hurtigbuffer (Strategi 2 i maskinvare)
 - Blokking (Strategi 1 i programvare)

Assosiative hurtigbuffer

TDT4260

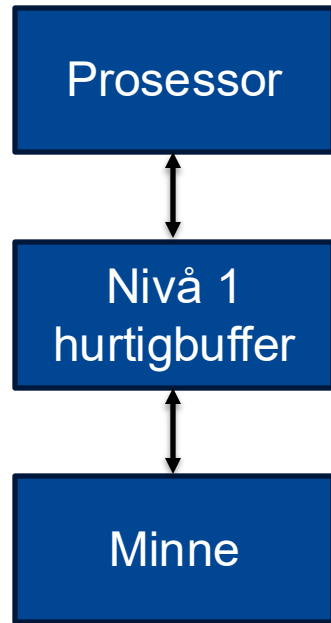
- Problem: I et direktetilordnet hurtigbuffer kan vi bare plassere én adresse på ett sted
 - Løsning: Gjør det mer fleksibelt hvor vi kan plassere hurtigbufferblokkene
- Nå kan vi velge mellom flere blokker når vi skal kaste ut noe
 - «Replacement policy»
 - Et typisk valg er «Least Recently Used (LRU)»
- Ekstremversjonen er et **fullassosiativt hurtigbuffer**
 - ... hvor en blokk kan plasseres hvor som helst
 - Dyrt: Vi må sammenlikne tag fra adressen med **alle** tags i hurtigbufferet



Figur 5.18: Et 4-veis settassosiativt hurtigbuffer

Flernivå hurtigbuffer

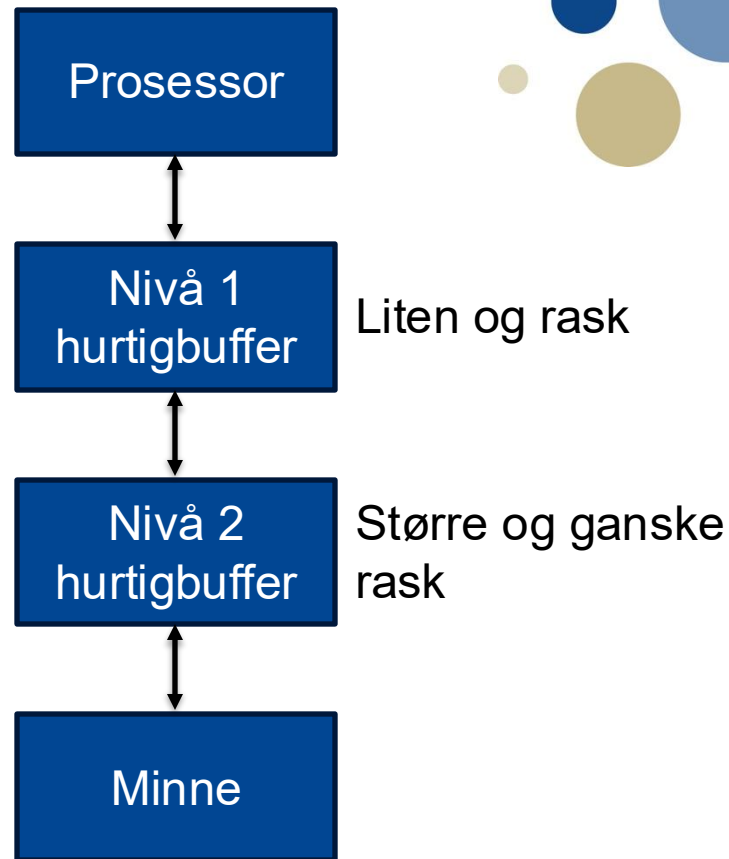
- Trefflatensen til et hurtigbuffer øker med størrelsen på hurtigbufferet
 - ... men aksess til hurtigbufferene i prosessoren er ofte på kritisk sti
 - ... og derfor må de være relativt små (for eksempel 32 KB)
- Løsning: Flere nivåer med hurtigbuffer
 - Reduserer latensen til bom fordi noen bom nå treffer i Nivå-2 hurtigbufferet



Flernivå hurtigbuffer

- Trefflatensen til et hurtigbuffer øker med størrelsen på hurtigbufferet
 - ... men aksess til hurtigbufferene i prosessoren er ofte på kritisk sti
 - ... og derfor må de være relativt små (for eksempel 32 KB)
- Løsning: Flere nivåer med hurtigbuffer
 - Reduserer latensen til bom fordi noen bom nå treffer i Nivå-2 hurtigbufferet

Menti!



Eksempel: Aksesstidsberegning

Oppgave 1

Nivå	Andel treff	Latens	Bidrag

Oppgave 2

Nivå	Andel treff	Latens	Bidrag

Eksempel: Aksesstidsberegning



Oppgave 3

Nivå	Andel treff	Latens	Bidrag

Blokking



TDT4200



- Ide: Vi reduserer antallet minneaksesser i programmet mellom hver aksess til samme data
 - Bedre lokalitet

Eksempel: Matrisemultiplikasjon

$$C = AB$$

```
for i in range(n):  
    for j in range(n):  
        for k in range(n):  
            C[i][j] +=  
                A[i][k] * B[k][j]
```



Tema 6.3: Virtuelt minne (Kapittel 5.6 og 5.7)

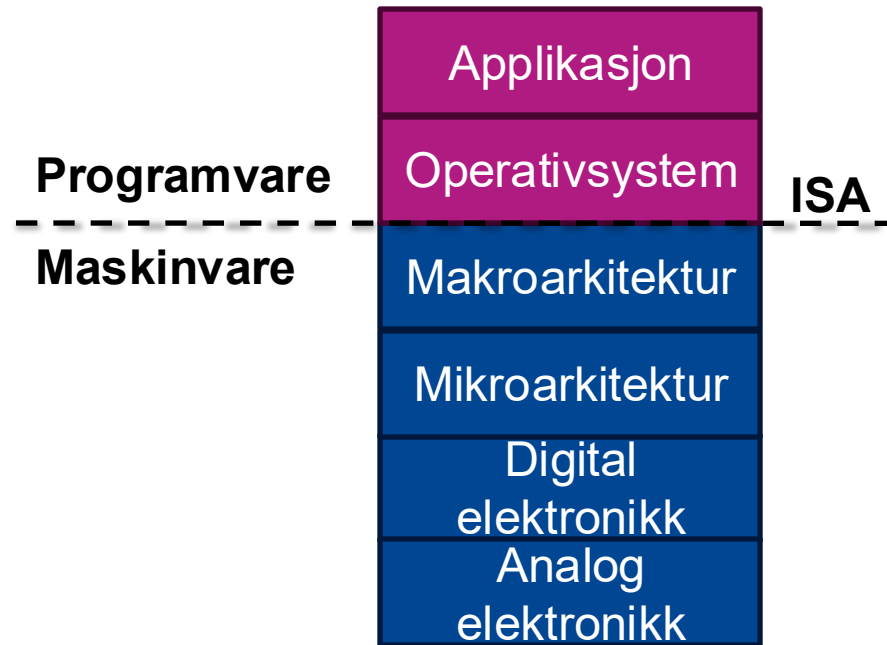
VIRTUELT MINNE

Læringsutbytte T6.3

- Studenten skal kunne forklare hvordan **virtuelle adresser oversettes til fysiske adresser**, inkludert arbeidsfordelingen mellom maskinvare og programvare.
- Studenten skal kunne forklare hvordan et «**Translation Lookaside Buffer (TLB)**» brukes til å implementere rask adresseoversettelse.
- Studenten skal kunne forklare hvordan virtuelt minne kan brukes til å **beskytte prosesser fra hverandres minneaksesser**.
- Studenten skal kunne forklare hvordan **sidefeil** håndteres.
- Studenten skal kunne forklare hva en **virtuell maskin** er.

Abstraksjonslag i datamaskinen

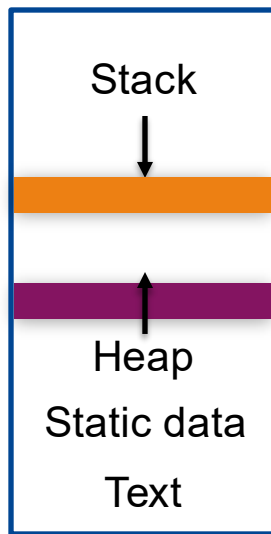
- Så langt har vi nesten utelukkende hatt et maskinvareperspektiv
 - Vi har enten løst rene maskinvareproblemer
 - ... eller vi har fått programvare til å gjøre jobben vår enklere (for eksempel kompilatoren)
- Datamaskinen må også bidra til å løse problemer på programvarenivå
 - Eksempel: Presise avbrudd
 - Nå: Virtuelt minne (og virtuelle maskiner)



Datamaskiner kjører typisk flere programmer samtidig

- ... men i våre prosessorer kan alle programmer skrive hvor som helst i minnet
 - Dette er både upraktisk og usikkert
- Løsning: Virtuelt minne
 - Operativsystemet bestemmer over det fysiske minnet
- Vi må oversette hver bidige minneadresse
 - ... som vil bli alt for tregt uten maskinvarestøtte

Programmets adresserom
(virtuelle adresser)



Adresseoversettelse

Datamaskinens minne
(fysiske adresser)



Sider og sidetabellen

- Vi deler inn minnet i sider («pages») (typisk på 4KiB)
- Operativsystemet bruker **sidetabellen** til å tilordne virtuelle sider til fysiske sider
 - Sidetabellen gir også informasjon om hvilke prosesser som har lov til å skrive og/eller lese denne siden
 - **Prosess** = Et kjørende program med tilstand (for eksempel PC)
 - Sidetabellen er lagret i minnet og kan bare endres av operativsystemet

Programmets adresserom
(virtuelle adresser)



Datamaskinens minne
(fysiske adresser)



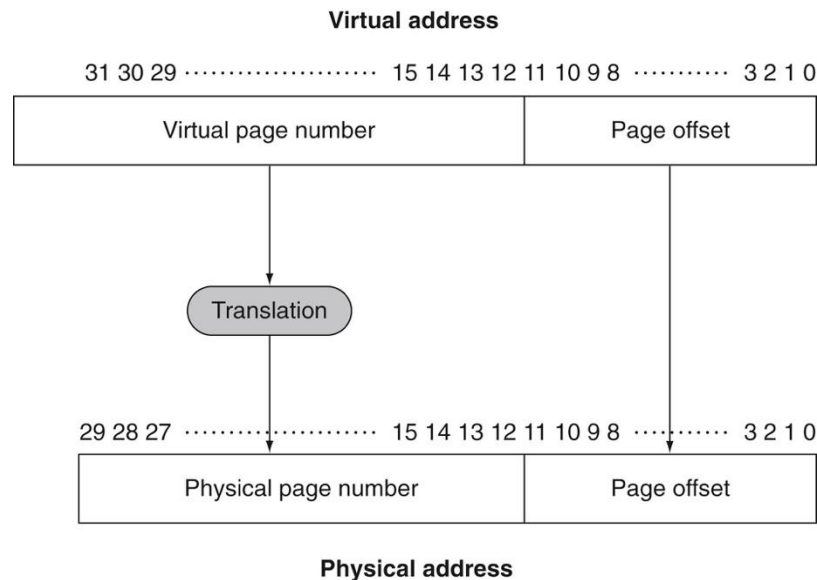
Sidetabell

v2 → f1

v1 → f2

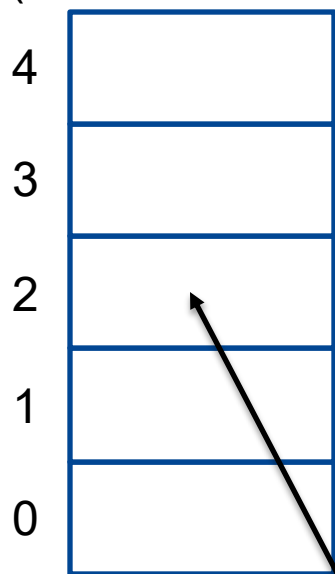
Adresseoversettelse

- Akkurat som med byte offset i hurtigbuffer vil de minst signifikante bitene i adressen gå til verdier inni siden
 - Kalles «page offset»
- Resten av adressen angir sidennummeret
 - ... og når vi oversetter adresser bytter vi ut det virtuelle sidennummeret med det fysiske sidennummeret gitt i sidetabellen



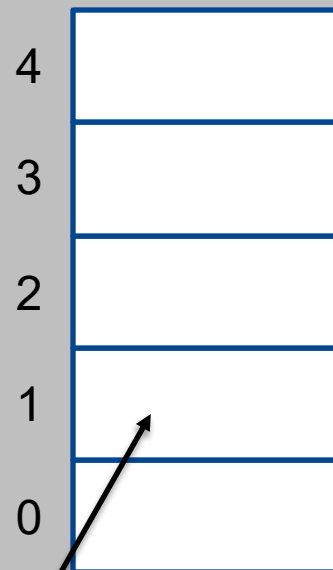
Minnesystem med virtuelt minne

**Adresserommet
til prosess a**
(virtuelle adresser)

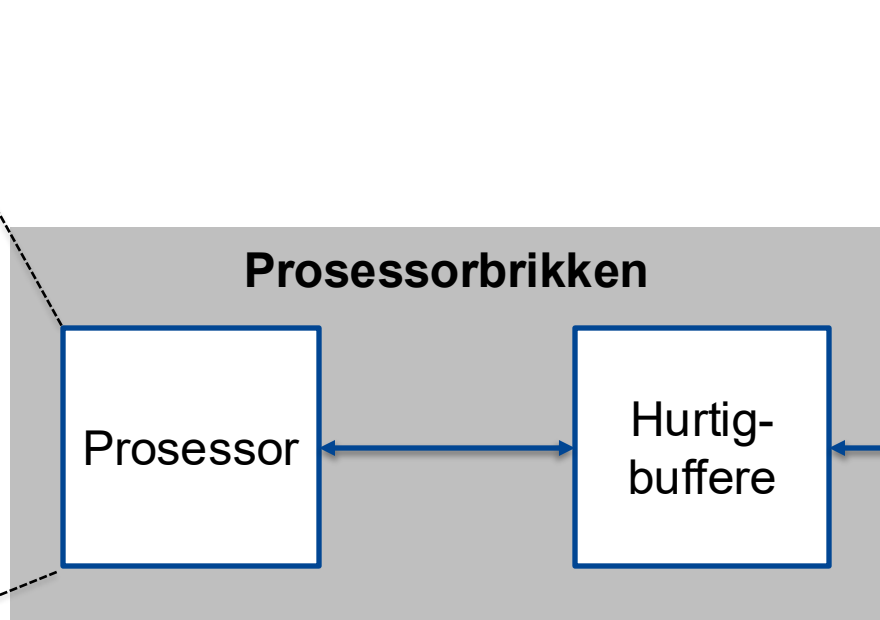


Virtuell side nummer 2

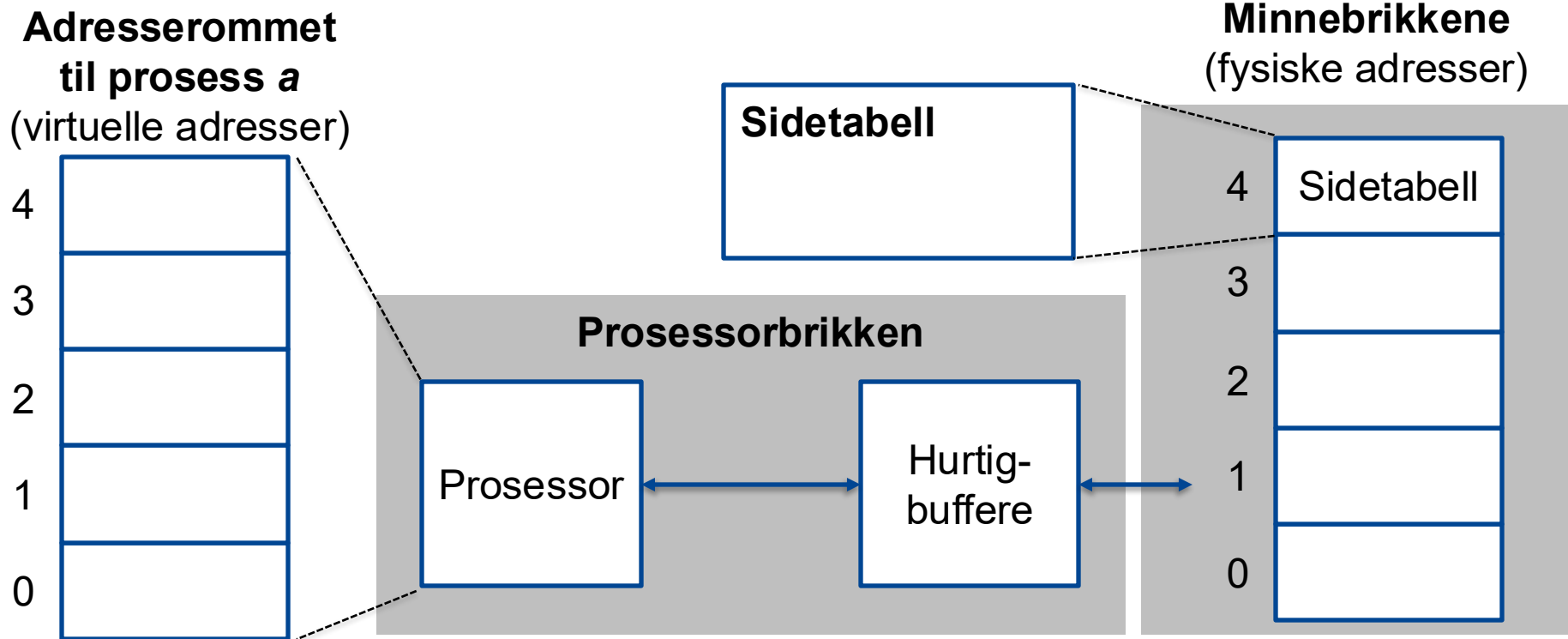
Minnebrikkene
(fysiske adresser)



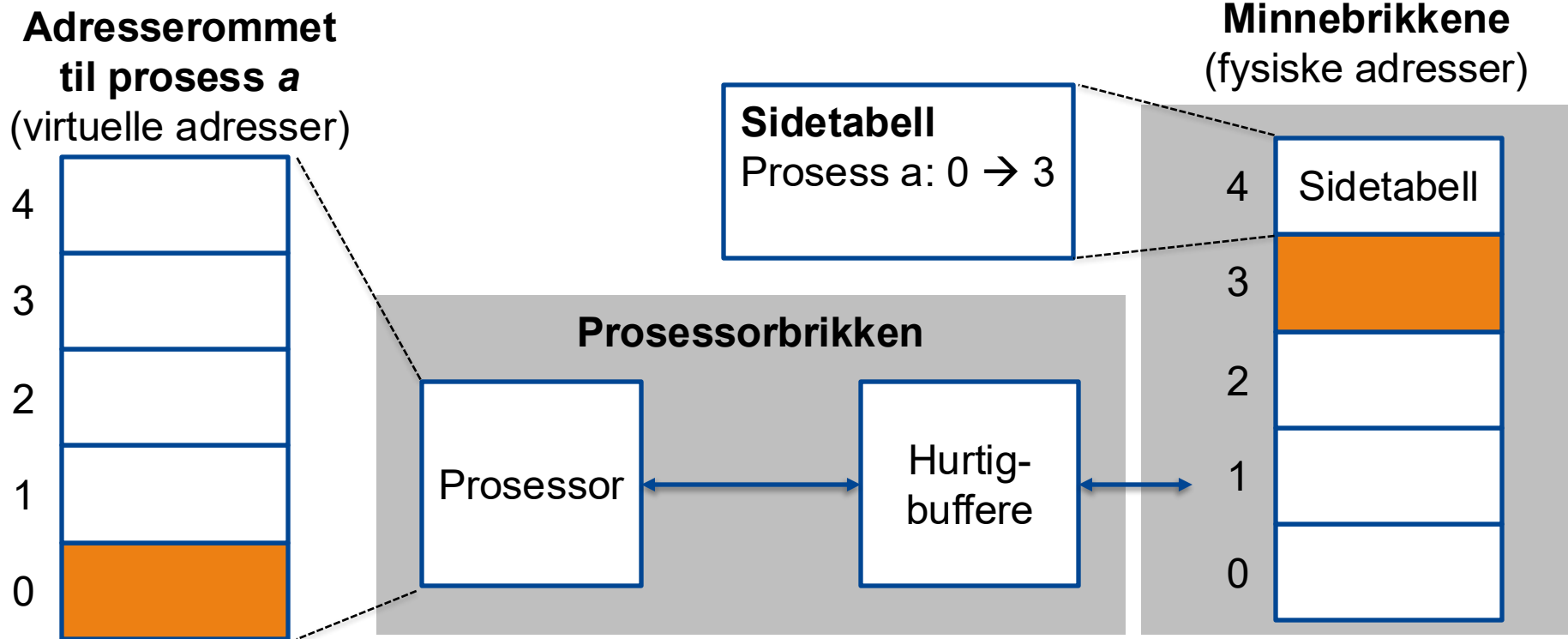
Fysisk side nummer 1



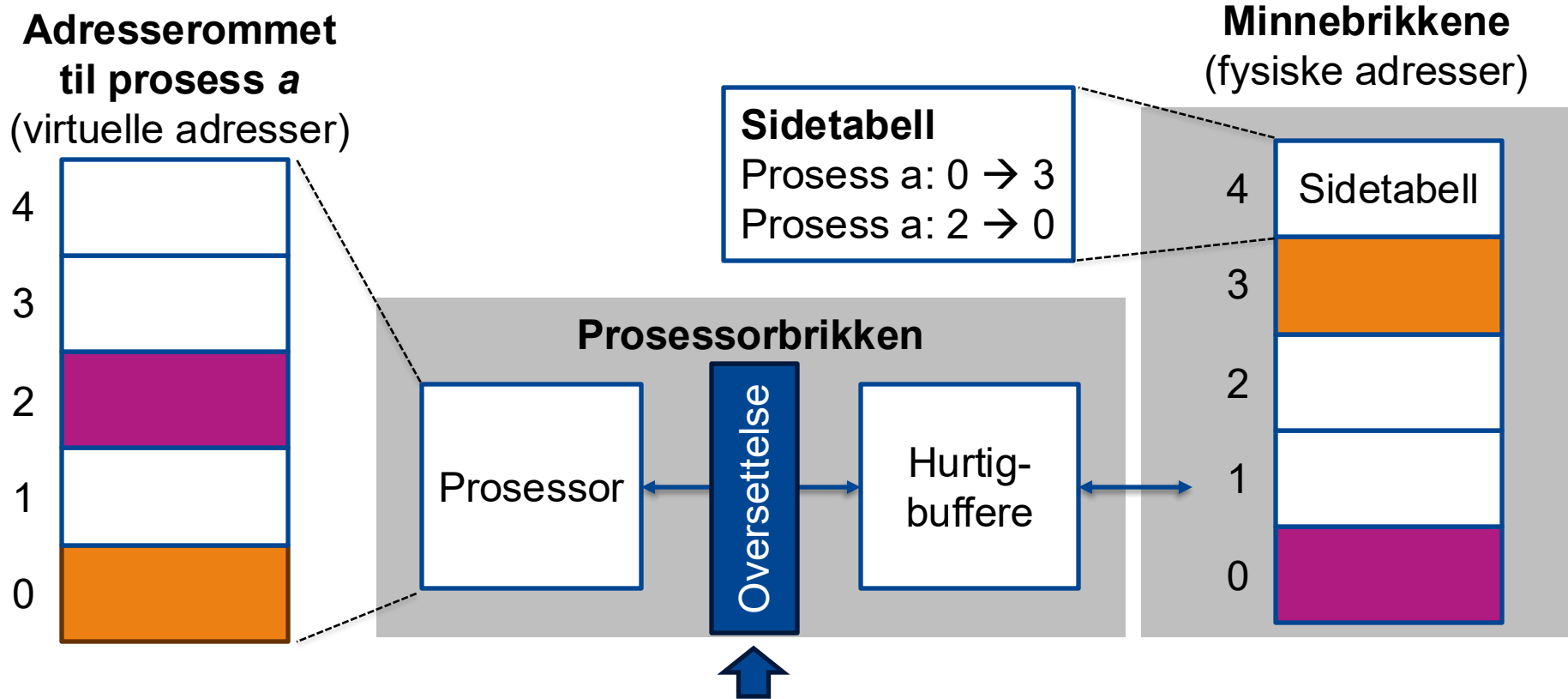
Minnesystem med virtuelt minne



Minnesystem med virtuelt minne



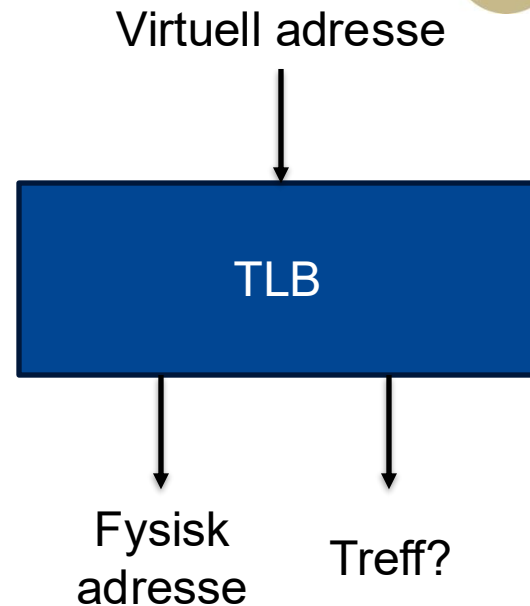
Minnesystem med virtuelt minne



Problem: Sidetabellen er i minnet men oversettelsen må gjøres nær prosessoren!

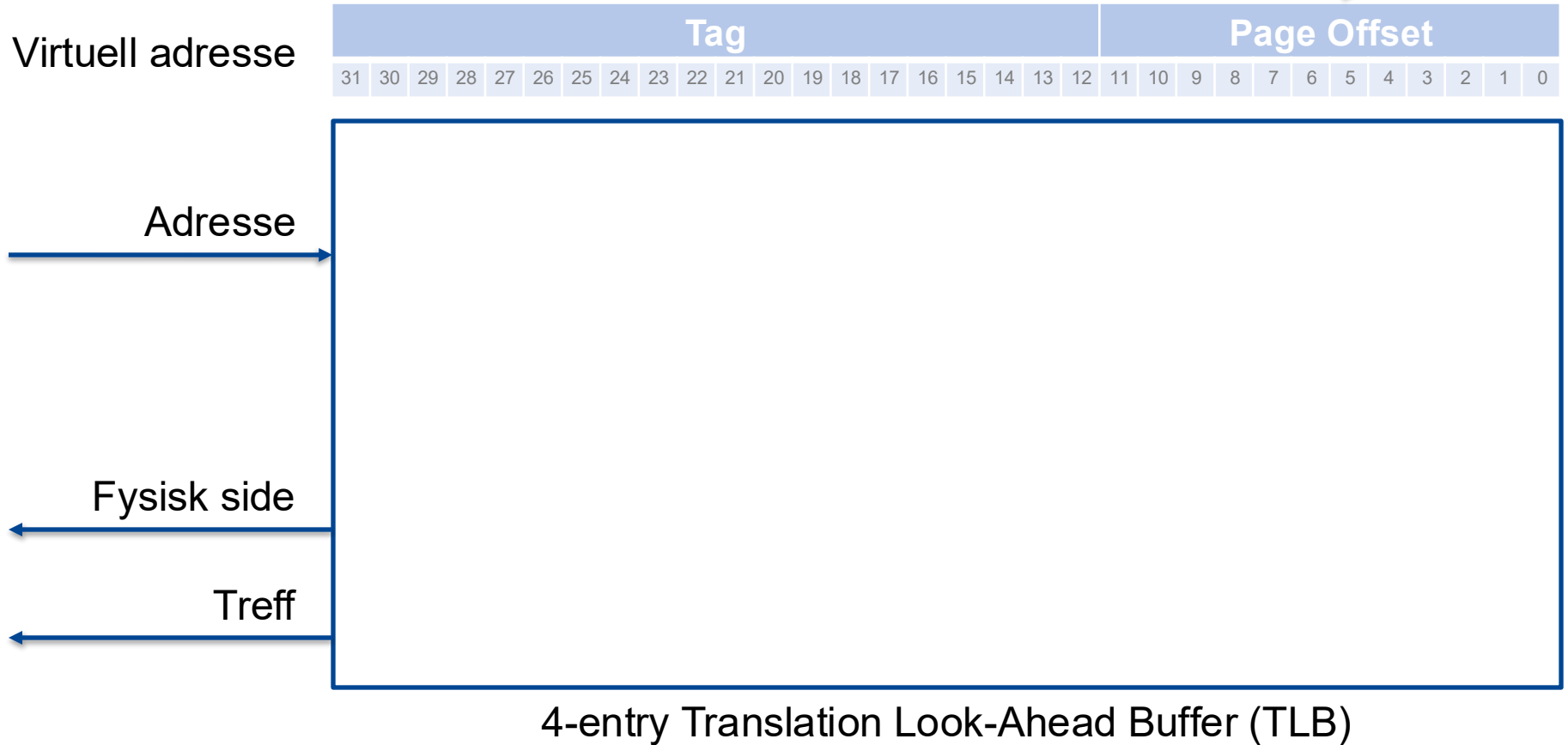
Translation Lookaside Buffer (TLB)

- Løsning: Et hurtigbuffer for adresseoversettelser
 - Kalles av historiske årsaker «Translation Lookaside Buffer (TLB)»
 - Vi har typisk en TLB for instruksjoner og en for data (for å ikke lage en strukturfare)
- Bom i TLB er dyrt
 - ... fordi vi må hente oversettelsen fra sidetabellen
 - Vi må også håndtere bom i TLB før vi kan starte minneforespørselen
 - ... fordi ellers vet vi ikke hvilken adresse vi skal aksessere
- Optimalisering
 - 1: Gjør TLBen fullassosiativ [øker treffrate]
 - 2: Legg til maskinvarestøtte for å slå opp i sidetabellen («Page table walker») [reduserer kostnad ved bom]



Eksempel: Fullassociativ TLB

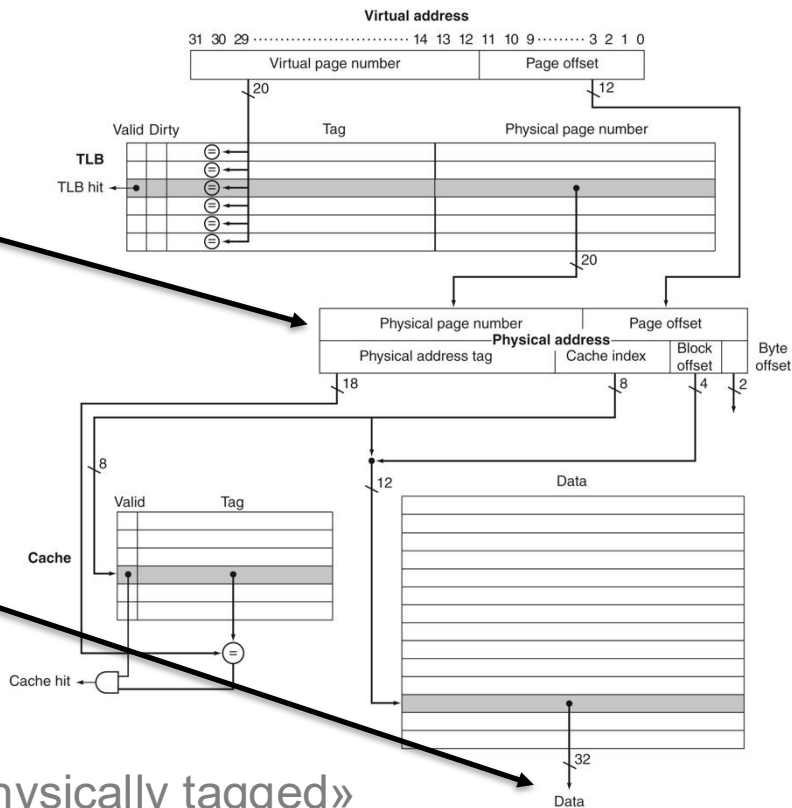
4 KiB sider
($2^{12} = 4096$)



Eksempel: Intrinsity FastMATH TLB

Vi slår opp i TLBen først for å få tak i den fysiske adressen

... så slår vi opp i hurtigbufferet for å få tak i verdien



Optimalisering: «Virtually indexed, physically tagged»
→ gjør det mulig å slå opp i TLB og hurtigbufferet i parallel

Beskyttelse («Protection»)

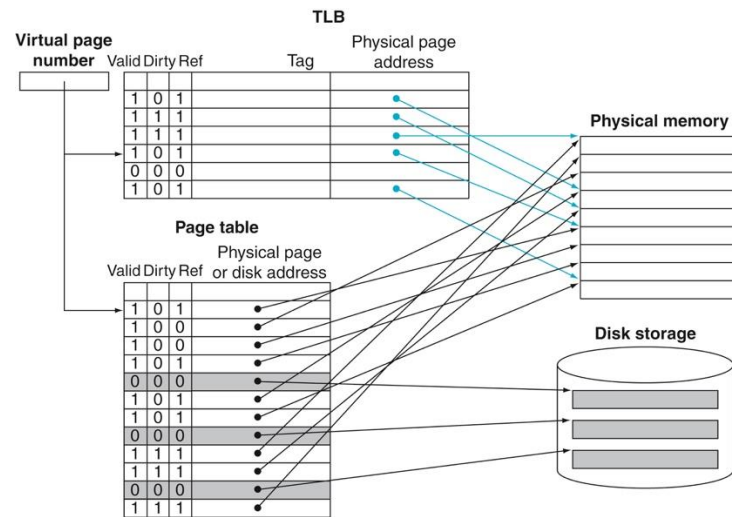
- Vi må isolere minneområdene til prosessene fra hverandre (feil og villet handling)
- Vi trenger følgende funksjonalitet for å implementere beskyttelse:
 - (Minst) to systemmodi
 - **Brukermodus** («user mode») og **systemmodus** («system mode» eller «kernel mode»)
 - **Privilegerte instruksjoner**: Genererer unntak hvis de kjøres i brukermodus
 - Tilstand i prosessoren som kan leses i brukermodus men kun skrives i systemmodus
 - Modusbit: Er vi i systemmodus eller ikke?
 - Sidetabellpeker: Peker til sidetabellen til prosessen som kjører akkurat nå
 - TLBene
 - Kunne bytte mellom brukermodus og systemmodus
 - Brukermodus til systemmodus: **Systemkall** («System call»), instruksjonen `ecall` i RISC-V
 - Systemmodus til brukermodus: **Systemretur**, instruksjonen `sret` i RISC-V
- Bytte mellom prosesser kalles et **kontekstbytte** («context switch»)
 - Operativsystemet lagrer all tilstanden til prosessen slik at den kan starte igjen som om ingenting har skjedd (PC, registerverdier, sidetabell, etc.) → *Tenk: Funksjonskall for prosesser*
 - Operativsystemet er en privilegert prosess (en prosess som kan gjøre ting som vanlige prosesser ikke kan)

Hvordan oppdateres sidetabellen?

- **Sidefeil** oppstår når siden vi aksesserer ikke er i minnet
 - «Swap space»: Et sted på disk/Flash der operativsystemet kan lagre sidene til prosesser
 - Sider legges typisk i minnet når de blir aksessert (og ikke når de blir allokert)

- Prosess *a* utfører en load-instruksjon men siden er ikke i sidetabellen → **en sidefeil** («*page fault*»)

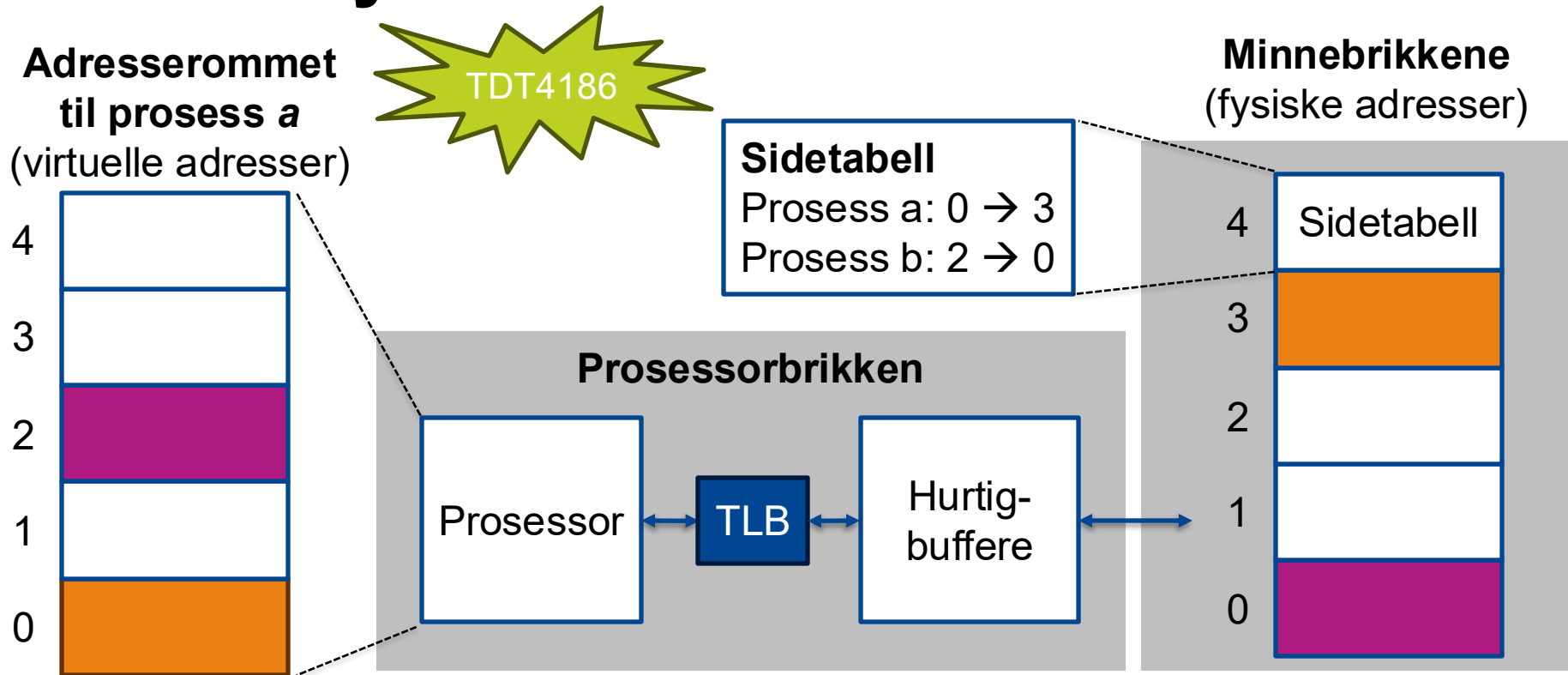
1. Prosessoren fullfører alle instruksjoner *som er eldre enn load-instruksjonen* og generer et **unntak**
 - Adressen i PC peker på load-instruksjonen
2. Operativsystemet håndterer **unntaket**
 - a. Bytter til systemmodus og lagrer **tilstanden** til Prosess *a* (inkludert PC og registrene x1 til x31)
 - b. Setter inn den manglende siden i minnet og oppdaterer sidetabellen
 - c. Legger inn den nye tilordningen i TLBen
 - d. Skriver tilbake **tilstanden** til Prosess *a*
 - e. Bytter til brukermodus og returner kontroll til Prosess *a*
3. Prosessoren utfører load-instruksjonen
 - Den vil nå treffe i TLBen!



Sidefeil er kjempedyrt
(tar typisk tusenvis av klokkesyker)

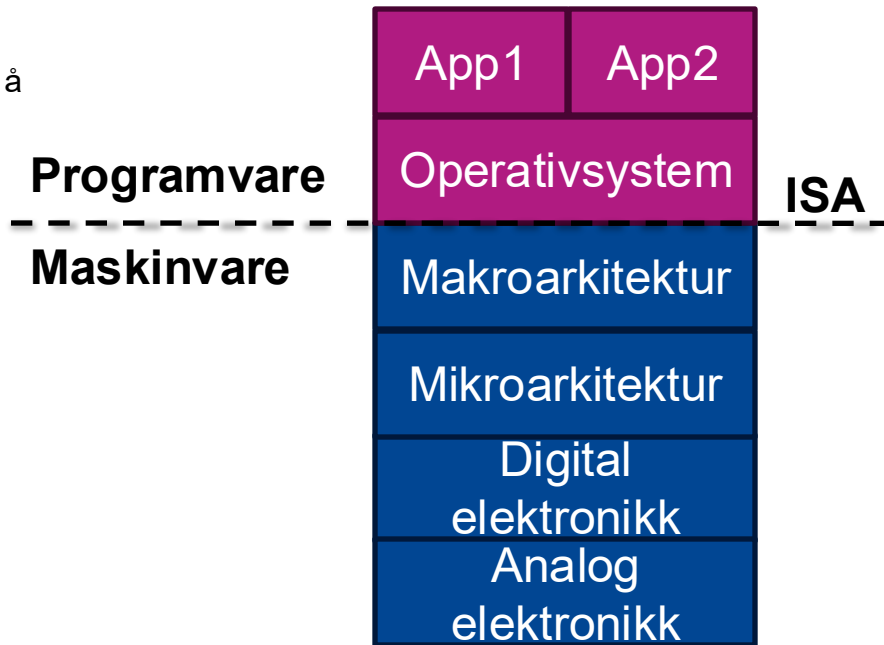
- Dette ville vært mye vanskeligere uten **presise unntak**!

Minnesystem med virtuelt minne



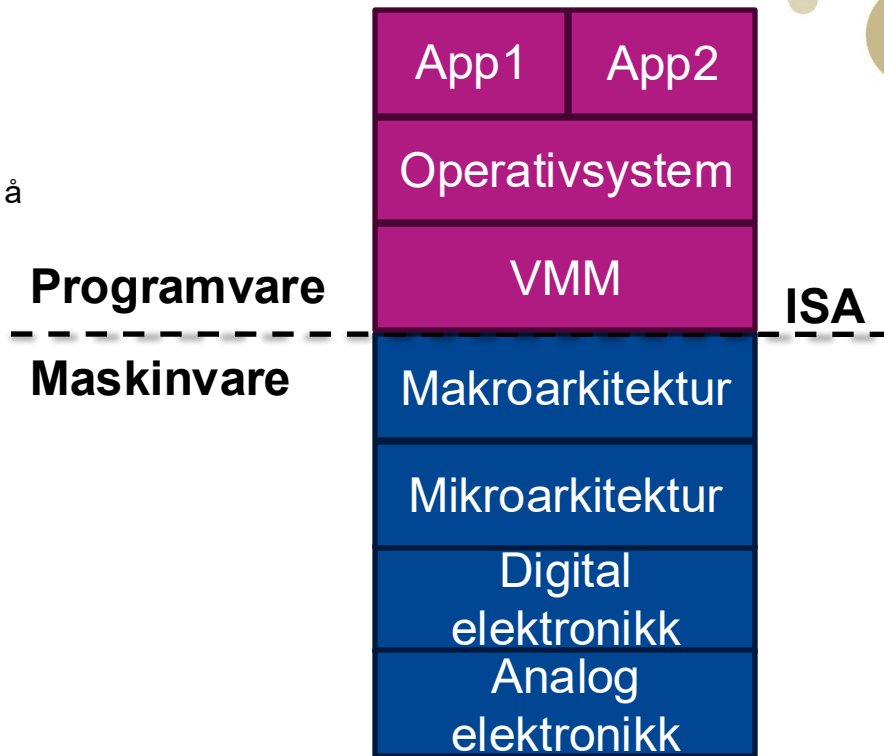
Virtuelle maskiner

- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**



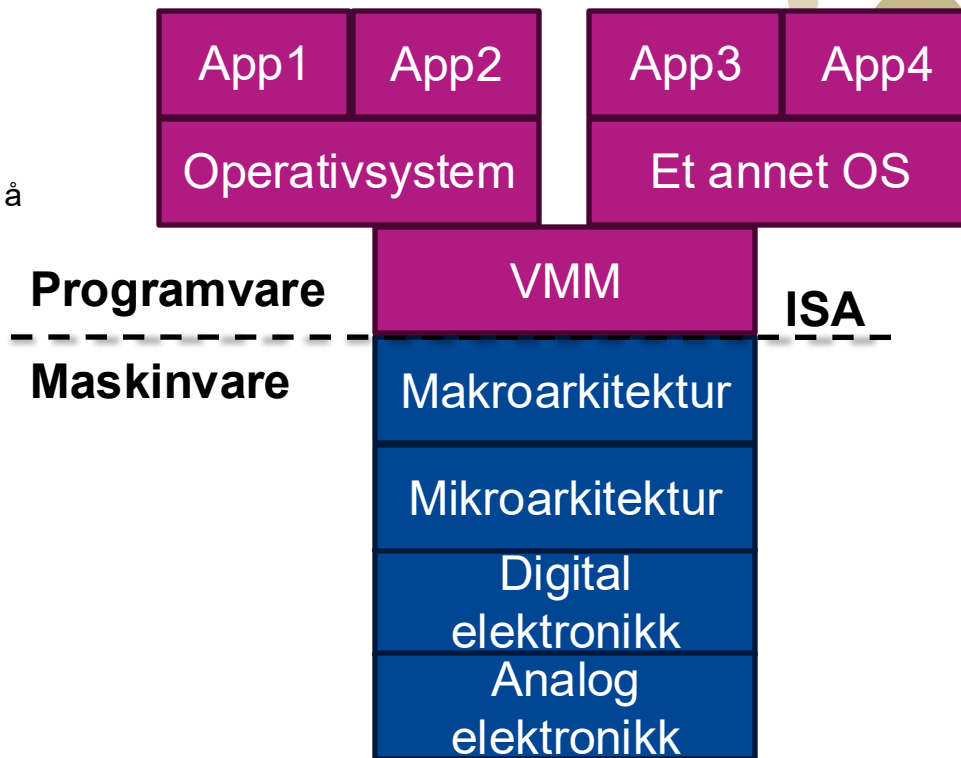
Virtuelle maskiner

- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**
- Nytt abstraksjonslag: «Virtual Machine Monitor (VMM)» eller «hypervisor»



Virtuelle maskiner

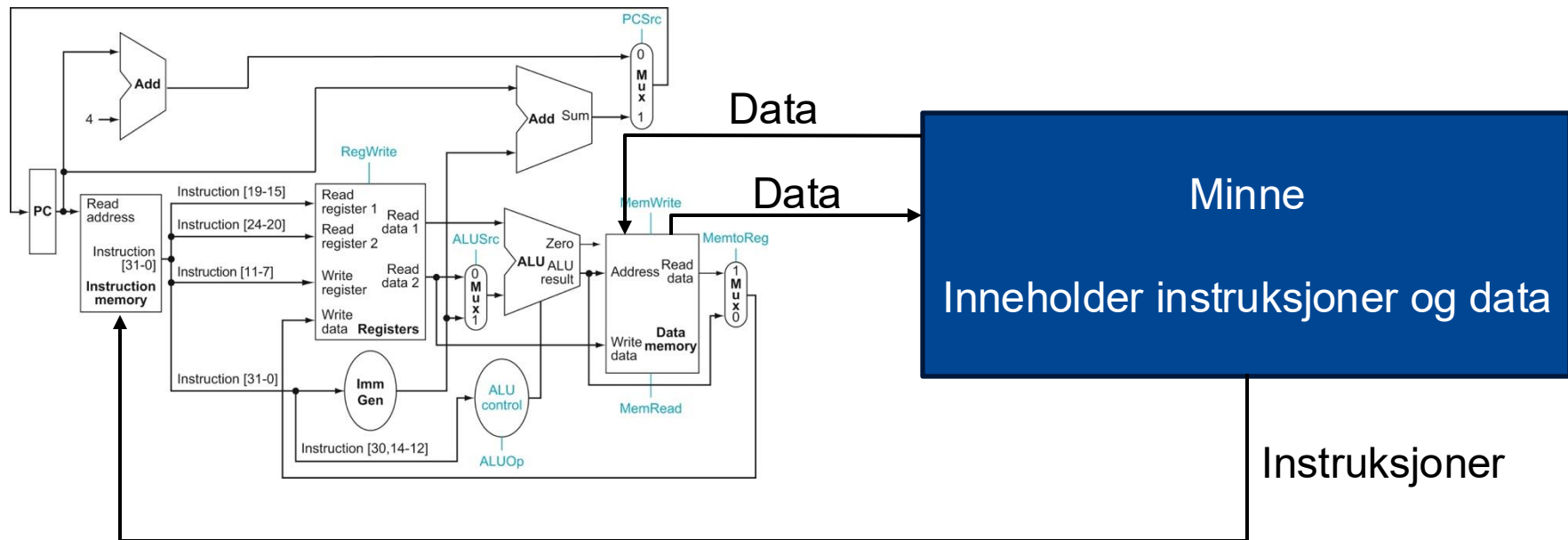
- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**
- Nytt abstraksjonslag: «Virtual Machine Monitor (VMM)» eller «hypervisor»
- Mye brukt i skyinfrastrukturer
 - ... fordi hver kunde kan sette opp sitt programvaremiljø som så kan tilordnes fysiske maskinressurser ved behov (f. eks. Amazon Web Services)





OPPSUMMERING

Prinsippet om lagrede program



Prinsippet om lagrede program

Magien er borte!

Proessorbrikken

Ofte flere nivåer med hurtigbuffere (SRAM)

